

Descripción de la empresa y problema de negocio

FleetLogix es una **empresa de transporte y logística** que opera una flota de 200 vehículos realizando entregas de última milla en 5 ciudades principales. La empresa ha estado operando con sistemas legacy y hojas de cálculo, pero necesita modernizar su infraestructura de datos para competir en el mercado actual.

Propuesta de solución

Para abordar esta problemática, se propone desarrollar una solución integral de datos dividida en **cuatro etapas consecutivas, denominadas avances**. Cada uno aborda una parte específica de la modernización de la infraestructura de FleetLogix.

Avance 1: Base de datos relacional: establece las bases de la nueva infraestructura de datos a partir del análisis del modelo relacional y la construcción de una base operacional poblada con datos sintéticos.

Avance 2: Análisis y optimización SQL: transforma los datos almacenados en información útil para el negocio mediante la ejecución y análisis de consultas SQL, el estudio de sus planes de ejecución y la aplicación de índices para mejorar el rendimiento de la base de datos.

Avance 3: Data Warehouse y pipeline ETL: incorpora una capa analítica mediante la implementación de un modelo dimensional en Snowflake y el desarrollo de procesos ETL en Python para extraer, transformar y cargar información desde PostgreSQL.

Avance 4: Arquitectura cloud en AWS: plantea la modernización de la infraestructura mediante el diseño y análisis de una arquitectura en AWS para recibir y procesar información de la flota en tiempo real. La propuesta incorpora servicios para procesamiento de eventos, almacenamiento, monitoreo, seguridad y migración de PostgreSQL hacia la nube.

Tech Stack utilizado

- PostgreSQL 15+
- Python 3.10+
- Jupyter Notebooks
- DBeaver o pgAdmin
- Snowflake
- AWS
- Git y GitHub

Control de versiones

Todo el desarrollo del Proyecto Integrador se gestiona mediante Git y GitHub para mantener el control de versiones. El repositorio reúne los scripts, la documentación y los archivos generados durante los distintos avances, permitiendo conservar un historial de los cambios realizados a medida que evoluciona el proyecto.

Repositorio del proyecto: [FleetLogix - GitHub](#)

Avance 1

Problema de negocio

FleetLogix necesita migrar de sistemas legacy y hojas de cálculo hacia una base de datos robusta que permita gestionar operaciones logísticas a gran escala.

Esta primera etapa plantea construir una base operacional que represente las principales entidades del negocio y sus relaciones. Para ello, se parte de un modelo relacional que debe ser analizado y documentado.

El modelo también debe poblarse con un volumen significativo de datos sintéticos realistas e interrelacionados, garantizando su calidad, integridad y consistencia temporal para los análisis posteriores.

Recursos proporcionados

Para el desarrollo del primer avance se proporcionan los siguientes recursos:

- **fleetlogix_db_schema.sql**: script SQL que contiene la definición del modelo relacional de FleetLogix y sus seis tablas principales.
- **A1-01_data_generation_estudiantes.py**: script base en Python destinado a la generación y carga de datos sintéticos utilizando Faker.
- **FleetLogix_guia_configuracion_inicial.pdf**: documento con las instrucciones y requisitos necesarios para preparar el entorno de trabajo del proyecto.

Desarrollo

El primer avance se desarrolla de manera progresiva, comenzando por la preparación del entorno de trabajo y continuando con el análisis del modelo relacional, la generación masiva de datos sintéticos y, finalmente, la validación de la información obtenida.

El trabajo se divide en cuatro etapas principales:

- **Configuración del entorno**: preparación de PostgreSQL, Python y las dependencias necesarias para ejecutar el proyecto.
- **Análisis del modelo relacional**: estudio de las seis tablas proporcionadas, sus relaciones, constraints, patrones de negocio y posibles mejoras.
- **Generación y carga de datos sintéticos**: adaptación y ejecución del script Python encargado de poblar las tablas respetando las reglas del negocio.
- **Control de calidad**: validación de integridad referencial, consistencia temporal y coherencia de los datos generados.

1. Configuración del entorno

Siguiendo *FleetLogix_guia_configuracion_inicial.pdf*, se utiliza **PostgreSQL** como sistema gestor de la base de datos y **Python** para los procesos de generación y tratamiento de datos. Se crea la base fleetlogix y se configura su acceso para permitir posteriormente la conexión desde Python.

Para administrar y consultar la base se utiliza **DBeaver**, desde donde se ejecutan los scripts SQL, se inspeccionan las tablas y se verifican los resultados obtenidos.

Finalmente, se instalan las dependencias de Python requeridas por el proyecto: psycopg2-binary, pandas, numpy, Faker, tabulate, Jupyter y notebook.

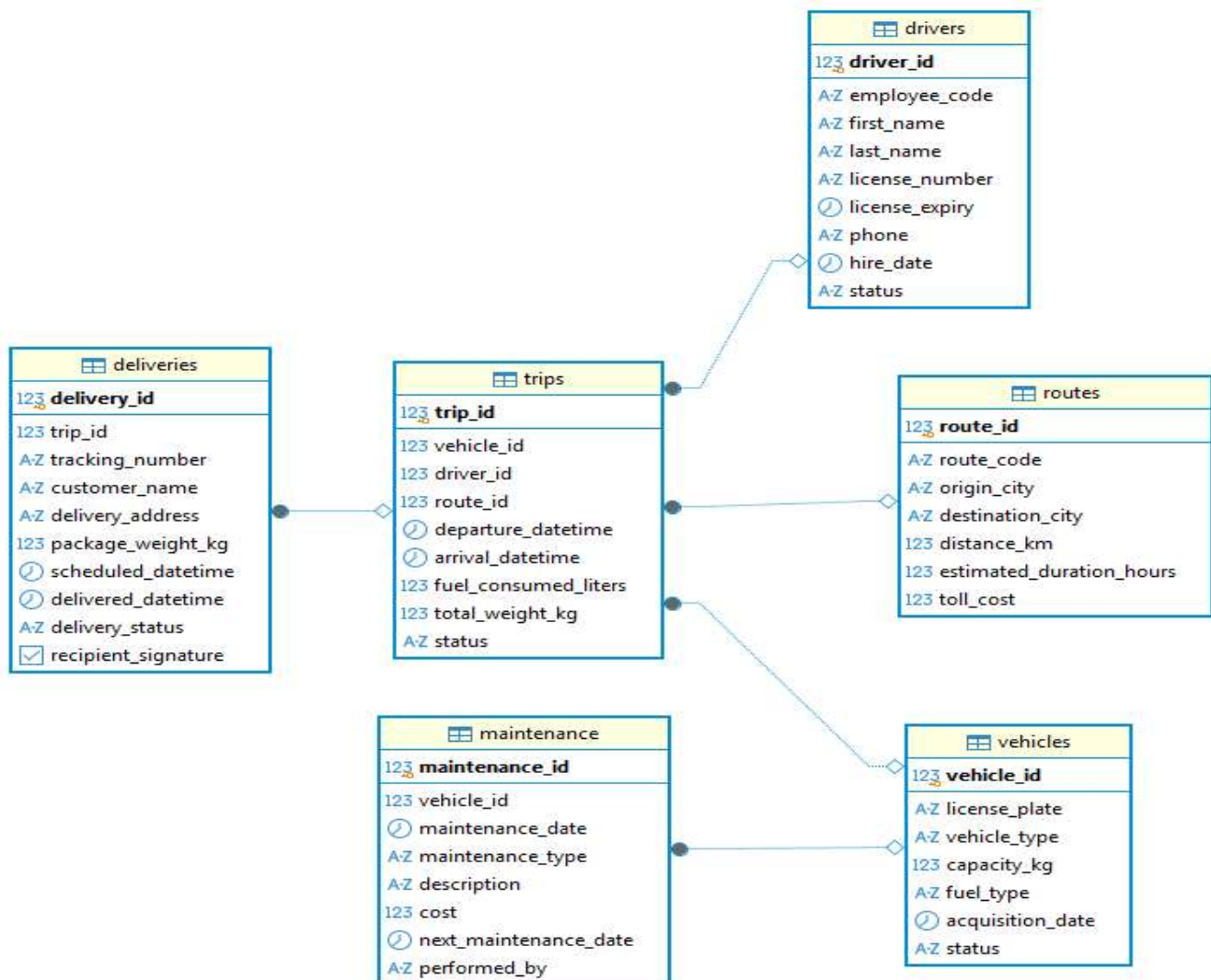
2. Implementación y análisis del modelo relacional

Una vez configurado el entorno, se implementó el modelo relacional proporcionado ejecutando el script fleetlogix_db_schema.sql sobre la base de datos fleetlogix. Como resultado, se crearon las seis tablas que representan las principales entidades de la operación: vehicles, drivers, routes, trips, deliveries y maintenance.

La estructura permite centralizar la información de la flota, los conductores y las rutas, vinculándola con los viajes, las entregas y el historial de mantenimiento de los vehículos.

2.1 Diagrama entidad-relación

A partir del modelo implementado se generó el siguiente diagrama entidad-relación (ER), que permite visualizar las tablas, sus atributos y las relaciones definidas entre ellas.



2.2 Análisis de las tablas

El modelo relacional está compuesto por seis tablas que representan las principales entidades y procesos de la operación logística. A continuación, se describe la función de cada una y su estructura.

- La tabla **vehicles** almacena la información de los vehículos que componen la flota de FleetLogix y sus principales características operativas.

Campo	Tipo de dato	Restricciones	Descripción
vehicle_id	SERIAL	PRIMARY KEY	Identificador único del vehículo.
license_plate	VARCHAR(20)	UNIQUE, NOT NULL	Patente del vehículo.
vehicle_type	VARCHAR(50)	NOT NULL	Tipo de vehículo.
capacity_kg	DECIMAL(10,2)	—	Capacidad de carga en kilogramos.
fuel_type	VARCHAR(20)	—	Tipo de combustible utilizado.
acquisition_date	DATE	—	Fecha de adquisición del vehículo.
status	VARCHAR(20)	DEFAULT 'active'	Estado operativo del vehículo.

- La tabla **drivers** almacena la información de los conductores de FleetLogix, incluyendo sus datos de identificación, licencia y situación laboral.

Campo	Tipo de dato	Restricciones	Descripción
driver_id	SERIAL	PRIMARY KEY	Identificador único del conductor.
employee_code	VARCHAR(20)	UNIQUE, NOT NULL	Código único asignado al empleado.
first_name	VARCHAR(50)	NOT NULL	Nombre del conductor.
last_name	VARCHAR(50)	NOT NULL	Apellido del conductor.
license_number	VARCHAR(50)	UNIQUE, NOT NULL	Número único de licencia de conducir.
license_expiry	DATE	—	Fecha de vencimiento de la licencia.
phone	VARCHAR(20)	—	Número de teléfono del conductor.
hire_date	DATE	—	Fecha de contratación.
status	VARCHAR(20)	DEFAULT 'active'	Estado laboral u operativo del conductor.

- La tabla **routes** almacena las rutas utilizadas por FleetLogix, definiendo las ciudades de origen y destino junto con información relevante para la planificación de los viajes.

Campo	Tipo de dato	Restricciones	Descripción
route_id	SERIAL	PRIMARY KEY	Identificador único de la ruta.
route_code	VARCHAR(20)	UNIQUE, NOT NULL	Código único asignado a la ruta.
origin_city	VARCHAR(100)	NOT NULL	Ciudad de origen de la ruta.
destination_city	VARCHAR(100)	NOT NULL	Ciudad de destino de la ruta.
distance_km	DECIMAL(10,2)	—	Distancia de la ruta en kilómetros.
estimated_duration_hours	DECIMAL(5,2)	—	Duración estimada del recorrido en horas.
toll_cost	DECIMAL(10,2)	DEFAULT 0	Costo de peajes asociado a la ruta.

- La tabla **trips** registra los viajes realizados por FleetLogix, vinculando cada operación con el vehículo, conductor y ruta correspondientes, además de almacenar información sobre tiempos, carga y consumo de combustible.

Campo	Tipo de dato	Restricciones	Descripción
trip_id	SERIAL	PRIMARY KEY	Identificador único del viaje.
vehicle_id	INTEGER	FOREIGN KEY	Vehículo asignado al viaje.
driver_id	INTEGER	FOREIGN KEY	Conductor asignado al viaje.
route_id	INTEGER	FOREIGN KEY	Ruta correspondiente al viaje.
departure_datetime	TIMESTAMP	NOT NULL	Fecha y hora de salida.
arrival_datetime	TIMESTAMP	—	Fecha y hora de llegada.
fuel_consumed_liters	DECIMAL(10,2)	—	Combustible consumido durante el viaje.
total_weight_kg	DECIMAL(10,2)	—	Peso total transportado en kg.
status	VARCHAR(20)	DEFAULT 'in_progress'	Estado del viaje.

- La tabla **deliveries** registra las entregas asociadas a los viajes de FleetLogix, incluyendo información del destinatario, paquete, programación y estado de la entrega.

Campo	Tipo de dato	Restricciones	Descripción
delivery_id	SERIAL	PRIMARY KEY	Identificador único de la entrega.
trip_id	INTEGER	FOREIGN KEY	Viaje al que pertenece la entrega.
tracking_number	VARCHAR(50)	UNIQUE, NOT NULL	Número único de seguimiento.
customer_name	VARCHAR(100)	NOT NULL	Nombre del cliente.
delivery_address	TEXT	NOT NULL	Dirección de entrega.

package_weight_kg	DECIMAL(10,2)	—	Peso del paquete en kilogramos.
scheduled_datetime	TIMESTAMP	—	Fecha y hora programada para la entrega.
delivered_datetime	TIMESTAMP	—	Fecha y hora en que se realizó la entrega.
delivery_status	VARCHAR(20)	DEFAULT 'pending'	Estado de la entrega.
recipient_signature	BOOLEAN	DEFAULT FALSE	Indica si se registró la firma del destinatario.

- La tabla ***maintenance*** registra los mantenimientos realizados a los vehículos de FleetLogix, incluyendo el tipo de intervención, fechas, costos y responsable.

Campo	Tipo de dato	Restricciones	Descripción
maintenance_id	SERIAL	PRIMARY KEY	Identificador único del mantenimiento.
vehicle_id	INTEGER	FOREIGN KEY	Vehículo al que corresponde el mantenimiento.
maintenance_date	DATE	NOT NULL	Fecha en que se realizó el mantenimiento.
maintenance_type	VARCHAR(100)	NOT NULL	Tipo de mantenimiento realizado.
description	TEXT	—	Descripción del trabajo realizado.
cost	DECIMAL(10,2)	—	Costo del mantenimiento.
next_maintenance_date	DATE	—	Fecha prevista para el próximo mantenimiento.
performed_by	VARCHAR(100)	—	Responsable de realizar el mantenimiento.

2.3 Relaciones del modelo

El modelo establece cinco relaciones mediante claves foráneas, permitiendo conectar los datos operativos y mantener la integridad referencial entre las tablas.

Relación	Clave foránea	Cardinalidad	Descripción
vehicles → trips	trips.vehicle_id	1:N	Un vehículo puede estar asociado a múltiples viajes, mientras que cada viaje referencia un vehículo.
drivers → trips	trips.driver_id	1:N	Un conductor puede realizar múltiples viajes, mientras que cada viaje referencia un conductor.
routes → trips	trips.route_id	1:N	Una ruta puede utilizarse en múltiples viajes, mientras que cada viaje referencia una ruta.
trips → deliveries	deliveries.trip_id	1:N	Un viaje puede incluir múltiples entregas, mientras que cada entrega pertenece a un viaje.
vehicles → maintenance	maintenance.vehicle_id	1:N	Un vehículo puede registrar múltiples mantenimientos, mientras que cada mantenimiento corresponde a un vehículo.

Estas relaciones permiten vincular los viajes con los recursos utilizados y las operaciones que dependen de ellos. “***trips***” ocupa una posición central en el modelo al relacionar vehículos, conductores y rutas, y servir a su vez como referencia para las entregas.

2.4 Constraints del modelo

El modelo utiliza diferentes constraints para garantizar la identificación de los registros, evitar duplicados y mantener la integridad de las relaciones entre las tablas.

Constraint	Aplicación en el modelo	Función
PRIMARY KEY	vehicle_id, driver_id, route_id, trip_id, delivery_id, maintenance_id	Identifica de manera única cada registro de las seis tablas.

FOREIGN KEY	trips.vehicle_id, trips.driver_id, trips.route_id, deliveries.trip_id, maintenance.vehicle_id	Mantiene la integridad referencial, evitando referencias a registros inexistentes.
UNIQUE	vehicles.license_plate, drivers.employee_code, drivers.license_number, routes.route_code, deliveries.tracking_number	Evita duplicados en atributos que deben ser únicos dentro del sistema.
NOT NULL	Aplicado a campos obligatorios de identificación y operación	Impide almacenar registros sin determinados datos considerados necesarios por el modelo.
DEFAULT	Estados de vehículos, conductores, viajes y entregas; costo de peajes y firma del destinatario	Asigna automáticamente un valor inicial cuando el campo no se especifica al insertar un registro.

En conjunto, estos constraints permiten mantener la consistencia de la información almacenada, garantizando la identificación única de los registros, la integridad de las relaciones entre las tablas y el cumplimiento de los campos definidos como obligatorios.

2.5 Patrones de negocio implícitos

La estructura del modelo permite identificar distintos patrones propios de la operación logística de FleetLogix.

Cada viaje vincula un vehículo, un conductor y una ruta, reflejando que una operación de transporte requiere la asignación conjunta de estos tres recursos. Esta relación permite conocer qué unidad fue utilizada, quién realizó el viaje y qué recorrido se efectuó.

Las entregas dependen de los viajes, permitiendo que una misma operación agrupe múltiples entregas y manteniendo la trazabilidad entre los pedidos y el recorrido realizado.

El modelo contempla variables operativas como la capacidad de los vehículos, el peso transportado, la distancia de las rutas, la duración estimada y el consumo de combustible. Estos atributos permiten representar restricciones y características propias de la operación logística.

Los registros de mantenimiento se asocian directamente a los vehículos, permitiendo conservar el historial de intervenciones de cada unidad.

Finalmente, las fechas y estados permiten representar la evolución temporal de la operación y diferenciar situaciones como vehículos activos, viajes en progreso o entregas pendientes y completadas.

2.6 Posibles mejoras del modelo

El análisis del modelo permite identificar mejoras orientadas a reforzar la calidad y consistencia de los datos, sin modificar la estructura base utilizada en el proyecto.

- **Validación de valores numéricos:** Incorporar restricciones CHECK para evitar valores que no tienen sentido operativo. Por ejemplo, la capacidad de un vehículo, el peso de una entrega, la distancia de una ruta o el costo de un mantenimiento no deberían admitir valores negativos.
- **Control de estados:** Los campos de estado están definidos como texto y aceptan cualquier valor. Limitar las opciones disponibles evita registros inconsistentes y garantiza que vehículos, conductores, viajes y entregas utilicen estados previamente definidos.
- **Coherencia temporal:** Las fechas deben respetar el orden lógico de cada proceso. La llegada de un viaje debe ser posterior a su salida y la fecha del próximo mantenimiento debe ser posterior al mantenimiento realizado.
- **Campos obligatorios:** Algunos datos relevantes actualmente admiten valores nulos. La capacidad del vehículo, la distancia y duración estimada de las rutas y el vencimiento de la licencia del conductor deberían registrarse obligatoriamente para poder aplicar correctamente las reglas operativas.
- **Condiciones operativas:** La existencia de un vehículo o conductor no significa necesariamente que esté habilitado para realizar un viaje. La asignación debe considerar el estado del vehículo y la vigencia de la licencia del conductor.

- **Compatibilidad entre conductor y vehículo:** Incorporar el tipo o categoría de licencia del conductor (license_type) permite verificar que cada conductor esté legalmente habilitado para manejar el vehículo que se le asigna.

3. Generación y carga de datos sintéticos

La generación de datos se realiza mediante el script **A1-01_data_generation_estudiantes.py**, diseñado para simular una operación logística coherente en Colombia. Los valores no se generan de forma independiente: cada tabla utiliza información de las entidades relacionadas para construir registros consistentes.

El proceso debe generar 200 vehículos, 400 conductores, 50 rutas, 100.000 viajes, 400.000 entregas y 5.000 registros de mantenimiento, alcanzando un volumen total superior a 505.000 registros.

3.1 Resultados de la primera ejecución

Una vez completadas las secciones pendientes del script, se realizó una primera ejecución completa y se verificó la cantidad de registros almacenados en cada tabla. Los resultados obtenidos fueron los siguientes:

Tabla	Registros
vehicles	200
drivers	400
routes	48
trips	100.000
deliveries	400.001
maintenance	4.920

Si bien el volumen total supera los 505.000 registros, routes, deliveries y maintenance no alcanzaron la cantidad esperada, por lo que fue necesario revisar la lógica de generación de estas tablas para su ajuste.

3.2 Ajustes realizados al script

A partir de los resultados obtenidos en la primera ejecución, se revisó la lógica de generación para corregir las diferencias detectadas y asegurar la consistencia de los datos.

Generación de rutas

El script generaba 48 rutas debido a la cantidad de recorridos asignados a cada combinación de ciudades. Se ajustó la distribución incorporando una tercera alternativa para los recorridos entre Medellín y Cali, en ambos sentidos. De esta forma, se alcanzaron las 50 rutas esperadas.

Generación de entregas

La condición de corte permitía generar una entrega adicional antes de finalizar el proceso, obteniendo 400.001 registros. Se modificó la condición para detener la generación al alcanzar el límite, obteniendo exactamente 400.000 entregas.

Período de los viajes

El intervalo utilizado para distribuir los viajes convertía los minutos a un número entero, reduciendo el período efectivo de generación. Se eliminó esta conversión para conservar los decimales y distribuir los 100.000 viajes a lo largo de los dos años completos de operación.

Distancias entre ciudades

La función de distancias ordenaba alfabéticamente las ciudades antes de realizar la búsqueda, pero algunas claves del diccionario no seguían ese mismo criterio. Se corrigieron las claves para obtener la distancia correspondiente independientemente del sentido del recorrido.

Registros de mantenimiento

La cantidad de mantenimientos se calcula individualmente para cada vehículo, generando uno por cada 20 viajes completos. Por este motivo se obtuvieron 4.913 registros. Los viajes restantes de cada vehículo no justifican un nuevo mantenimiento, por lo que se mantuvo esta lógica sin forzar el total a 5.000.

3.3 Resultados de la ejecución final

Una vez aplicadas las correcciones, se ejecutó nuevamente el proceso completo de generación. Los resultados obtenidos fueron:

Tabla	Registros generados
vehicles	200
drivers	400
routes	50
trips	100.000
deliveries	400.000
maintenance	4.913
Total	505.563

3.4 Lógica de generación y justificación de tablas

La generación de datos sigue la estructura relacional del modelo, utilizando información de las tablas previamente creadas para mantener coherencia entre los registros.

Vehículos

La tabla vehicles genera la flota utilizando cuatro tipos de vehículos: Camión Grande, Camión Mediano, Van y Motocicleta. Cada uno posee una capacidad de carga, tipo de combustible, fecha de adquisición y estado. La capacidad se utiliza posteriormente para limitar el peso transportado en los viajes.

Conductores

La tabla drivers genera información ficticia de los conductores, incluyendo código de empleado, datos personales, licencia, fecha de vencimiento, contratación y estado. Para generar los viajes se seleccionan conductores con estado activo.

Rutas

La tabla routes genera recorridos entre Bogotá, Medellín, Cali, Barranquilla y Cartagena. Cada ruta contiene origen, destino, distancia, duración estimada y costo de peajes. La distancia y duración son utilizadas posteriormente para calcular características de los viajes.

Viajes

La tabla trips relaciona las tres entidades anteriores. Mediante generate_trips() se seleccionan vehículos activos, conductores activos y rutas existentes para generar 100.000 viajes distribuidos a lo largo de dos años.

La duración del viaje toma como referencia la duración estimada de la ruta; el consumo de combustible se calcula según la distancia recorrida; y el peso transportado se genera entre el 40 % y el 90 % de la capacidad del vehículo. La hora de salida se determina mediante _get_hourly_distribution(), permitiendo representar distintos niveles de actividad durante el día.

Entregas

La tabla deliveries se genera a partir de los viajes existentes. Cada viaje contiene entre 2 y 6 entregas, siendo 4 la cantidad más probable. El peso de los paquetes se distribuye utilizando aproximadamente el 95 % del peso transportado y sus horarios se calculan en relación con los tiempos del viaje.

Mantenimiento

La tabla maintenance utiliza el historial de viajes de cada vehículo para generar aproximadamente un mantenimiento por cada 20 viajes completos. Se contemplan diferentes intervenciones, como cambio de aceite, revisión de frenos, cambio de llantas, mantenimiento general, revisión de motor y alineación y balanceo.

3.5 Método generate_trips() y distribución horaria

El método generate_trips() es responsable de construir el historial de viajes. Para hacerlo, consulta las tablas vehicles, drivers y routes, obteniendo los vehículos activos junto con su capacidad, los conductores activos y las rutas disponibles. A partir de estos datos genera cada registro de trips, manteniendo las relaciones entre las tablas.

Los viajes se distribuyen progresivamente a lo largo de aproximadamente 730 días. Para cada uno se selecciona un vehículo, un conductor y una ruta. La información seleccionada se utiliza para calcular variables relacionadas: la duración se basa en el tiempo estimado de la ruta, el consumo de combustible en la distancia recorrida y el peso transportado en la capacidad del vehículo. La fecha de llegada se obtiene a partir de la salida y de la duración calculada.

El método auxiliar _get_hourly_distribution() se utiliza para evitar que las horas de salida tengan la misma probabilidad durante todo el día. Se establece una actividad menor durante las horas de baja demanda, una mayor actividad entre las 6:00 y las 20:00 y dos franjas de mayor concentración: entre las 8:00 y las 12:00 y entre las 14:00 y las 18:00.

De esta manera, generate_trips() combina las relaciones del modelo con una distribución temporal más realista, generando viajes coherentes tanto con los recursos disponibles como con los horarios habituales de una operación logística.

4. Control de Calidad

Una vez finalizada la generación y carga de los datos, se utiliza el script validacion_calidad_datos.sql para verificar la calidad y consistencia de la información almacenada en la base de datos.

El script realiza tres grupos de controles:

4.1 Cantidad de registros

Se verifica individualmente la cantidad de registros de cada tabla para comprobar los resultados obtenidos durante la generación.

4.2 Integridad referencial

Se comprueba que los viajes estén asociados a vehículos, conductores y rutas existentes; que las entregas pertenezcan a viajes registrados; y que los mantenimientos correspondan a vehículos existentes.

4.3 Consistencia temporal

Se verifica que la fecha y hora de llegada de cada viaje sea posterior a su fecha y hora de salida, permitiendo detectar posibles inconsistencias temporales.

Estos controles permiten validar los datos generados antes de utilizarlos en las etapas posteriores de análisis.

4.4 Resultados de la validación

La ejecución de validacion_calidad_datos.sql no detectó inconsistencias. Todos los controles de integridad referencial y consistencia temporal devolvieron 0 registros, indicando que no existen relaciones inválidas entre las tablas ni viajes cuya fecha de llegada sea anterior o igual a la fecha de salida.

Los conteos de las tablas también confirmaron los resultados obtenidos durante la generación: 200 vehículos, 400 conductores, 50 rutas, 100.000 viajes, 400.000 entregas y 4.913 registros de mantenimiento, alcanzando un total de 505.563 registros.

5. Mejoras futuras

El modelo y los datos generados permiten representar la operación principal de FleetLogix y constituyen una base para futuros análisis. A partir del trabajo realizado se identifican oportunidades de mejora tanto técnicas como orientadas al negocio.

5.1 Mejoras del modelo y generación de datos

- Incorporar el tipo de licencia del conductor y definir su compatibilidad con los distintos tipos de vehículos.
- Incorporar controles de disponibilidad de conductores para evitar la asignación del mismo conductor a viajes con horarios superpuestos.
- Refinar el cálculo del consumo de combustible considerando el tipo de vehículo, la distancia recorrida y la carga transportada.
- Mejorar la lógica de entregas demoradas para representar con mayor precisión el cumplimiento de los horarios programados.
- Ampliar las validaciones automáticas del script Python para cubrir todas las relaciones del modelo.
- Revisar los parámetros de generación de rutas y mantenimientos para permitir un mayor control sobre los volúmenes generados.

5.2 Mejoras orientadas al negocio

- Incorporar información sobre costos operativos de cada viaje para analizar la rentabilidad de rutas y operaciones.
- Registrar tarifas o ingresos asociados a las entregas para comparar ingresos y costos.
- Incorporar información geográfica más detallada para analizar recorridos, zonas de entrega y eficiencia de las rutas.
- Registrar causas de retrasos y entregas fallidas para identificar problemas recurrentes.
- Incorporar métricas de utilización de los vehículos para detectar unidades con baja utilización o sobrecarga operativa.
- Analizar el historial de mantenimiento junto con kilometraje, viajes realizados y costos para avanzar hacia estrategias de mantenimiento preventivo.
- Incorporar indicadores de desempeño de conductores, rutas y vehículos que permitan comparar eficiencia, cumplimiento y costos.

Avance 2

Problema de negocio

FleetLogix cuenta ahora con una base de datos operacional poblada y validada, pero necesita transformar los registros almacenados en información útil para analizar el desempeño de sus operaciones.

Esta segunda etapa se enfoca en utilizar consultas SQL de distinta complejidad para responder preguntas relacionadas con vehículos, conductores, rutas, viajes, entregas y mantenimientos.

Además del análisis de la información, se busca evaluar y optimizar el rendimiento de las consultas mediante el estudio de sus planes de ejecución y la implementación de índices, comparando los tiempos de respuesta antes y después de la optimización.

Recursos proporcionados

Para el desarrollo del segundo avance se proporcionan los siguientes recursos:

- **02_queries_analysis.sql:** script SQL que contiene las 12 consultas propuestas, organizadas según distintos niveles de complejidad, a partir de las cuales se seleccionan las consultas requeridas para el análisis.
- **A2-03_optimization_indexes.sql:** script SQL destinado a la creación de índices para optimizar el rendimiento de las consultas analizadas.

Desarrollo

El segundo avance parte de las consultas proporcionadas en *02_queries_analysis.sql*, orientadas a obtener información relevante sobre la operación de FleetLogix mediante distintos niveles de complejidad SQL.

A partir de las consultas seleccionadas, el trabajo comprende su ejecución y análisis, la identificación del problema de negocio que aborda cada una y el estudio de sus planes de ejecución mediante EXPLAIN ANALYZE.

Luego, se implementan cinco índices de optimización utilizando *A2-03_optimization_indexes.sql*, con el objetivo de evaluar su impacto sobre el rendimiento. Para ello, se comparan los tiempos de ejecución obtenidos antes y después de aplicar los índices.

El análisis detallado de las consultas seleccionadas se documenta de forma independiente en *Manual_Consultas_SQL.pdf*.

1. Ejecución y análisis de consultas SQL

El archivo *02_queries_analysis.sql* contiene 12 consultas SQL organizadas en distintos niveles de complejidad. Para el desarrollo del avance se requiere seleccionar y analizar un mínimo de ocho consultas, manteniendo una combinación que permita trabajar tanto operaciones básicas como técnicas de análisis más avanzadas.

La selección debe incluir las tres consultas básicas, las consultas 4 y 5, una consulta entre la 6 y la 8, la consulta 9 y una consulta entre la 10 y la 12.

Cada consulta seleccionada se ejecuta sobre la base de datos **fleetlogix** para comprobar su funcionamiento y analizar los resultados obtenidos. Su explicación individual, junto con el problema de negocio que busca resolver, se desarrolla en *Manual_Consultas_SQL.pdf*.

1.1 Query 1: Contar vehículos por tipo

Problema de negocio: Conocer la composición de la flota.

Funcionamiento de la consulta: La consulta agrupa los registros de la tabla `vehicles` según el tipo de vehículo (`vehicle_type`) y utiliza `COUNT(*)` para calcular la cantidad de unidades de cada categoría. Finalmente, ordena los resultados de mayor a menor cantidad.

Resultado obtenido:

vehicle_type	cantidad
Van	69
Camión Grande	60
Camión Mediano	51
Motocicleta	20

Análisis del resultado: La flota está compuesta por 200 vehículos. Las vans representan el grupo más numeroso, con 69 unidades (34,5 % de la flota), seguidas por los camiones grandes con 60 (30 %) y los camiones medianos con 51 (25,5 %). Las motocicletas presentan la menor participación, con 20 unidades (10 %).

El resultado permite conocer la distribución actual de la flota e identificar el peso relativo de cada tipo de vehículo, información que puede servir como punto de partida para analizar la disponibilidad y composición de los recursos de transporte.

1.2 Query 2: Listar conductores con licencia próxima a vencer

Problema de negocio: Prevenir problemas legales por licencias próximas a vencer.

Funcionamiento de la consulta: La consulta selecciona los conductores cuya licencia vence dentro de los próximos 30 días, utilizando `CURRENT_DATE` como referencia, y ordena los resultados por fecha de vencimiento. Al depender de la fecha actual, el resultado es dinámico; esta ejecución corresponde al 14/09/2026.

Resultado obtenido:

first_name	last_name	license_number	license_expiry
Daniel	Medina	3044426338	2026-09-15
Lorena	Díaz	6404582459	2026-10-02
Camilo	Álvarez	7048809022	2026-10-02
Joan	Muñoz	4047453184	2026-10-06
David	Espitia	2881937953	2026-10-10
Marlon	González	3099500153	2026-10-10

Análisis del resultado: Se identificaron 6 conductores con licencias próximas a vencer. El caso más inmediato es Daniel Medina, con vencimiento el 15/09/2026, mientras que los restantes se concentran entre el 2 y el 10 de octubre.

Esta información permite priorizar las renovaciones según su proximidad y reducir el riesgo de asignar operaciones a conductores con licencias vencidas.

1.3 Query 3: Total de viajes por estado

Problema de negocio: Monitorear el estado general de los viajes.

Funcionamiento de la consulta: La consulta agrupa los registros de la tabla trips según su estado (status) y utiliza COUNT(*) para calcular la cantidad de viajes correspondiente a cada categoría.

Resultado obtenido:

status	total_viajes
completed	99,929
in_progress	71

Análisis del resultado: De los 100.000 viajes registrados, 99.929 se encuentran completados y 71 permanecen en progreso, por lo que aproximadamente el 99,93 % corresponde a operaciones finalizadas.

El resultado permite obtener una visión general del estado de los viajes e identificar rápidamente el volumen de operaciones que todavía se encuentran en curso.

1.4 Query 4: Total de entregas por ciudad destino en los últimos 2 meses

Problema de negocio: Analizar la distribución reciente de la demanda entre las ciudades donde opera FleetLogix.

Funcionamiento de la consulta: La consulta relaciona rutas, viajes y entregas para calcular por ciudad de destino la cantidad de viajes, entregas y peso total transportado durante los últimos 2 meses. Al utilizar CURRENT_DATE para definir este período, los resultados son dinámicos; esta ejecución corresponde al 14/09/2026.

Resultado obtenido:

destination_city	total_viajes	total_entregas	peso_total_kg
Bogotá	1,886	7,557	3,189,771.8
Medellín	1,632	6,588	2,722,109.55
Cali	1,586	6,359	2,768,423.83
Cartagena	1,463	5,833	2,498,117.27
Barranquilla	1,437	5,734	2,460,375.73

Análisis del resultado: Bogotá concentra la mayor actividad del período, con 1.886 viajes y 7.557 entregas, además del mayor peso total transportado. Medellín ocupa el segundo lugar en cantidad de viajes y entregas,

mientras que Cali, aunque registra menos viajes que Medellín, presenta un peso total transportado ligeramente superior.

Los resultados permiten comparar el volumen operativo reciente entre las ciudades y detectar cuáles concentran una mayor demanda de transporte.

1.5 Query 5: Conductores activos con cantidad de viajes completados

Problema de negocio: Analizar la carga operativa de los conductores activos a partir de la cantidad de viajes asignados y completados.

Funcionamiento de la consulta: La consulta selecciona los conductores activos y relaciona sus registros con los viajes realizados. Para cada conductor calcula la cantidad total de viajes y cuántos de ellos fueron completados, ordenando los resultados según la cantidad de viajes completados.

Resultado obtenido: Debido al volumen de registros obtenidos, se muestran los primeros resultados:

driver_id	nombre_completo	license_expiry	viajes_totales	viajes_completados
341	Camilo Delgado	2029-01-29	311	310
7	Sandra Salamanca	2029-05-25	310	309
265	Elizabeth López	2028-02-02	308	308
347	Diego Barrios	2028-10-09	308	307
277	María Díaz	2028-11-23	306	306
57	Nancy Garcés	2029-01-12	306	306
253	María Vásquez	2029-06-05	306	305
327	Tania Calderón	2028-11-19	302	302
37	Humberto Cerón	2027-12-29	302	302
305	Stella García	2026-12-13	302	301

Análisis del resultado: Los resultados muestran una carga de trabajo relativamente alta entre los primeros conductores del listado. Camilo Delgado presenta la mayor cantidad de viajes, con 311 asignados y 310 completados, seguido por Sandra Salamanca con 310 viajes y 309 completados.

También se observan conductores, como Elizabeth López, María Díaz y Nancy Garcés, que tienen todos los viajes mostrados como completados. La comparación entre viajes totales y completados permite detectar aquellos conductores que todavía poseen viajes en otro estado y analizar la distribución de la actividad entre los conductores activos.

1.6 Query 6: Promedio de entregas por conductor en los últimos 6 meses

Problema de negocio: Analizar la actividad de los conductores a partir de la cantidad de viajes y entregas realizadas durante los últimos 6 meses.

Funcionamiento de la consulta: La consulta relaciona conductores, viajes y entregas para calcular por conductor el total de viajes y entregas, junto con el promedio de entregas por viaje y por día durante los últimos 6 meses. Al utilizar CURRENT_DATE para determinar el período analizado, los resultados son dinámicos; esta ejecución corresponde al 14/09/2026.

Resultado obtenido: Debido al volumen de registros obtenidos, se muestran los primeros resultados:

Driver id	conductor	Total viajes	Total entregas	Promedio entregas por viaje	Promedio entregas diarias
370	Felipe Mercado	79	341	4.32	1.89
250	Joan Muñoz	73	314	4.3	1.74
8	Camilo Mejía	71	305	4.3	1.69
293	José Salas	62	266	4.29	1.48

256	María Ramírez	69	295	4.28	1.64
310	Alba Vargas	62	265	4.27	1.47
313	Oswaldo Camargo	72	306	4.25	1.7
255	Fernando Arrieta	69	293	4.25	1.63
367	Ibeth Rico	51	217	4.25	1.21
15	Orlando Herrera	76	323	4.25	1.79

Análisis del resultado: Los resultados muestran que los conductores ubicados en las primeras posiciones presentan un promedio cercano a **4,3 entregas por viaje**. Felipe Mercado encabeza el listado con 4,32 entregas por viaje, seguido por Joan Muñoz y Camilo Mejía, ambos con 4,30.

Al observar las demás métricas, se aprecia que el volumen de actividad varía entre conductores. Por ejemplo, Felipe Mercado realizó 79 viajes y 341 entregas, mientras que Carmen Blanco registró 87 viajes y 361 entregas, alcanzando además uno de los mayores promedios diarios observados, con 2,01 entregas por día.

1.7 Query 7: Rutas con mayor consumo de combustible por kilómetro

Problema de negocio: Identificar las rutas que presentan un mayor consumo de combustible en relación con la distancia recorrida.

Funcionamiento de la consulta: La consulta analiza los viajes completados de cada ruta y calcula la cantidad de viajes realizados, el consumo promedio y total de combustible y los litros consumidos cada 100 kilómetros. Los resultados se ordenan según esta última métrica para mostrar las 10 rutas con mayor consumo relativo.

Resultado obtenido:

Route code	ruta	Distance km	Viajes realizados	Promedio combustible litros	Litros por 100km	Combustible total
R009	Bogotá -> Barranquilla	960.66	2,009	111.4324639124	11.6	223,867.82
R043	Cartagena -> Bogotá	1,068.17	1,963	123.7880234335	11.59	242,995.89
R025	Cali -> Bogotá	417.69	1,944	48.3498251029	11.58	93,992.06
R038	Barranquilla -> Cali	1,099.1	1,924	127.1073648649	11.56	244,554.57
R003	Bogotá -> Medellín	457.97	1,948	52.9270585216	11.56	103,101.91
R005	Bogotá -> Cali	489.81	2,010	56.5522587065	11.55	113,670.04
R045	Cartagena -> Medellín	660.84	2,042	76.3249902057	11.55	155,855.63
R016	Medellín -> Cali	402.44	2,026	46.477023692	11.55	94,162.45
R011	Bogotá -> Cartagena	1,026.98	2,042	118.5022967679	11.54	241,981.69
R019	Medellín -> Barranquilla	680.52	2,049	78.5636993655	11.54	160,977.02

Análisis del resultado: La ruta R009, Bogotá -> Barranquilla, presenta el mayor valor de litros_por_100km, con 11.6, seguida por R043, Cartagena -> Bogotá, con 11.59, y R025, Cali -> Bogotá, con 11.58.

Los valores de consumo relativo son muy similares entre las diez rutas, ya que se encuentran entre 11.54 y 11.6 litros por cada 100 km. En cambio, el combustible_total presenta diferencias más marcadas entre rutas. Por ejemplo, R038, Barranquilla -> Cali, registra el mayor consumo total del resultado con 244,554.57 litros, mientras que R025, Cali -> Bogotá, presenta 93,992.06 litros.

1.8 Query 8: Entregas retrasadas por día de la semana

Problema de negocio: Identificar si existen días de la semana con una mayor proporción de entregas retrasadas.

Funcionamiento de la consulta: La consulta analiza las entregas realizadas durante los últimos 90 días y las agrupa según el día de la semana. Para cada día calcula el total de entregas, la cantidad de entregas

retrasadas, el porcentaje de retrasos y la diferencia promedio en minutos. Al utilizar CURRENT_DATE para determinar el período analizado, los resultados son dinámicos; esta ejecución corresponde al 14/09/2026.

Resultado obtenido:

dia semana	num dia	total entregas	entregas retrasadas	porcentaje retrasos	minutos promedio diferencia
Sunday	0	6,554	671	10.24	12.5064083003
Monday	1	6,516	665	10.21	12.332412523
Tuesday	2	7,071	691	9.77	12.0106067034
Wednesday	3	7,242	726	10.02	12.5541286937
Thursday	4	7,128	687	9.64	11.2210998878
Friday	5	7,205	706	9.8	11.9576682859
Saturday	6	6,737	688	10.21	11.9671960813

Análisis del resultado: Los porcentajes de retraso son bastante similares a lo largo de la semana, ya que se encuentran entre 9.64 % y 10.24 %. El domingo presenta el porcentaje más alto, con 10.24 %, mientras que el jueves registra el menor, con 9.64 %.

En cantidad absoluta, el miércoles concentra el mayor número de entregas retrasadas, con 726, aunque también es el día con mayor cantidad total de entregas, con 7,242.

En conjunto, los resultados no muestran una diferencia marcada entre los días de la semana, ya que los porcentajes de retraso se mantienen próximos al 10 % en todos los casos.

1.9 Query 9: Análisis de costos de mantenimiento por tipo de vehículo

Problema de negocio: Comparar los costos de mantenimiento de los distintos tipos de vehículos en relación con su utilización.

Funcionamiento de la consulta: La consulta agrupa la información por tipo de vehículo y combina datos de vehículos, viajes y mantenimientos. Para cada categoría calcula la cantidad de vehículos, los viajes y kilómetros acumulados, el costo total de mantenimiento, el costo por kilómetro recorrido y el costo promedio por mantenimiento.

Resultado obtenido:

vehicle type	cantidad vehiculos	viajes totales	kilometros totales	costo total mantenimiento	costo por km	costo promedio por mantenimiento
Motocicleta	19	10,435	192,526,755.11	90,132,914,417.65	468.16	174,786,325.31
Camión Mediano	47	25,924	485,820,328.26	221,468,810,646.61	455.87	173,018,288.04
Van	62	34,410	648,537,537.88	294,147,113,474.74	453.55	173,448,138.29
Camión Grande	53	29,160	546,031,769.47	242,786,661,053.33	444.64	168,983,619.34

Análisis del resultado: Los resultados muestran que las motocicletas presentan el mayor costo de mantenimiento por kilómetro, con 468,16, seguidas por los camiones medianos con 455,87 y las vans con 453,55. Los camiones grandes registran el menor valor, con 444,64 por kilómetro.

En términos de costo total de mantenimiento, las vans presentan el valor más elevado, aunque también concentran la mayor cantidad de vehículos y kilómetros recorridos dentro del resultado. Por este motivo, el costo por kilómetro permite realizar una comparación más equilibrada entre los distintos tipos de vehículos que el costo total considerado de forma aislada.

1.10 Query 10: Ranking de conductores por eficiencia

Problema de negocio: Comparar el desempeño de los conductores considerando simultáneamente distintas métricas operativas.

Funcionamiento de la consulta: La consulta evalúa a los conductores a partir de la cantidad de viajes y entregas, el consumo de combustible cada 100 km y el porcentaje de puntualidad. Luego utiliza funciones de ranking para establecer posiciones según puntualidad, eficiencia y productividad, y combina esos rankings mediante un score promedio.

El análisis considera información de los últimos 3 meses. Al utilizar CURRENT_DATE para determinar este período, los resultados son dinámicos; esta ejecución corresponde al 14/09/2026.

Resultado obtenido:

nombre	viajes	entregas	consumo 100km	puntualidad pct	rank puntualidad	Rank eficiencia	rank productividad	score promedio
Carmen Blanco	45	193	10.94	50.78	49	27	5	27
Lucy Guevara	40	160	11.16	51.88	32	72	51	51.6666666667
Aurora García	42	171	11.1	49.12	86	52	29	55.6666666667
Wilfrido Sanabria	39	176	11.07	47.73	115	42	18	58.3333333333
Carolina Pineda	43	169	11.05	47.93	110	39	30	59.6666666667
Eduardo Suárez	34	135	10.59	52.59	21	5	161	62.3333333333
Marlon González	41	162	10.76	46.91	143	11	43	65.6666666667
Paola Buitrago	42	172	10.95	45.93	162	29	27	72.6666666667
Claudia Suárez	41	161	11.36	51.55	37	139	48	74.6666666667
Antonio Ortiz	49	195	11.5	51.79	33	195	3	77
Andrés Herrera	33	124	10.03	55.65	8	1	226	78.3333333333
Humberto Cerón	41	157	11.45	56.05	6	172	58	78.6666666667
María Gutiérrez	38	157	11.26	49.68	72	107	58	79
Antonio Buitrago	39	151	11.16	49.01	89	70	86	81.6666666667
María Pedraza	39	151	10.92	47.02	135	25	86	82
León Jiménez	31	140	11.24	52.86	16	101	133	83.3333333333
Elías Delgado	48	194	10.83	43.81	249	13	4	88.6666666667
Miguel López	45	184	11.37	47.83	113	142	12	89
Alberto Ochoa	38	144	11.09	47.92	111	45	112	89.3333333333
Gustavo Buitrago	38	161	11.5	52.17	29	196	48	91

Análisis del resultado: Carmen Blanco ocupa la primera posición del resultado, con un score promedio de 27, producto de las posiciones 49 en puntualidad, 27 en eficiencia y 5 en productividad. Esto muestra que su posición final surge de la combinación de las tres métricas y no de liderar necesariamente cada una por separado.

Esto se observa claramente al comparar otros casos. Andrés Herrera ocupa el primer puesto en eficiencia y el octavo en puntualidad, pero su posición 226 en productividad eleva su score promedio hasta 78.3333333333. Por otro lado, Antonio Ortiz presenta una productividad muy alta, con el puesto 3, pero ocupa el puesto 195 en eficiencia.

Por lo tanto, el resultado permite obtener una clasificación conjunta de los conductores considerando diferentes dimensiones de su desempeño operativo, en lugar de evaluarlos mediante una única métrica.

2. Análisis de planes de ejecución

Para evaluar el rendimiento de las consultas seleccionadas, se utiliza EXPLAIN ANALYZE, una herramienta de PostgreSQL que permite conocer cómo se ejecuta una consulta y cuánto tiempo tarda.

Los resultados obtenidos sirven como punto de referencia para posteriormente aplicar los índices de optimización y comparar si el rendimiento mejora.

Resultados de los planes de ejecución

Query 1: El plan muestra un Seq Scan sobre la tabla vehicles, procesando 200 registros. Luego utiliza HashAggregate para realizar la agrupación y Sort para ordenar el resultado.

La estimación del plan fue de 200 registros y la ejecución procesó también 200 registros. El Planning Time fue de 2.325 ms y el Execution Time de 0.160 ms

Query 2: El plan muestra un Seq Scan sobre la tabla drivers, procesando 400 registros y descartando 394 mediante el filtro de fecha. Luego utiliza Sort para ordenar los 6 registros obtenidos por license_expiry.

La estimación del plan fue de 7 registros y la ejecución obtuvo 6 registros. El Planning Time fue de 2.334 ms y el Execution Time de 0.130 ms.

Query 3: El plan muestra un Seq Scan sobre la tabla trips, procesando 100,000 registros. Luego utiliza HashAggregate para agrupar los viajes por estado y Sort para ordenar los 2 resultados obtenidos.

La estimación del plan fue de 100,000 registros y la ejecución procesó también 100,000 registros. El Planning Time fue de 1.655 ms y el Execution Time de 25.358 ms.

Query 4: El plan combina las tablas mediante Hash Join. Sobre deliveries realiza un Seq Scan, procesando 400,000 registros, mientras que sobre trips utiliza un Index Scan para obtener 8,054 registros correspondientes al período analizado. Luego utiliza GroupAggregate para agrupar los resultados y Sort para ordenarlos.

La estimación previa a la agrupación fue de 31,820 registros y la ejecución procesó 32,071 registros. El Planning Time fue de 0.328 ms y el Execution Time de 107.594 ms.

Query 5: El plan combina drivers y trips mediante un Hash Right Join. Realiza un Seq Scan sobre trips, procesando 100,000 registros, y otro sobre drivers, donde obtiene 373 conductores activos y descarta 27. Luego utiliza HashAggregate para agrupar los resultados y Sort para ordenarlos.

La estimación final fue de 124 registros, mientras que la ejecución obtuvo 373 registros. El Planning Time fue de 0.313 ms y el Execution Time de 48.132 ms.

Query 6: El plan combina deliveries, trips y drivers mediante Hash Join. Realiza un Seq Scan sobre deliveries, procesando 400,000 registros, y utiliza un Index Scan sobre trips, obteniendo 24,969 registros del período analizado. Luego utiliza GroupAggregate para agrupar los resultados y Sort para ordenarlos.

La estimación final fue de 133 registros, mientras que la ejecución obtuvo 373 registros. El Planning Time fue de 0.351 ms y el Execution Time de 185.459 ms.

Query 7: El plan combina trips y routes mediante un Hash Join. Realiza un Seq Scan sobre trips, procesando 99,929 registros y descartando 71 mediante el filtro, y otro sobre routes, procesando 50 registros. Luego utiliza HashAggregate, Sort y finalmente Limit para obtener las 10 rutas del resultado.

La estimación de la agrupación fue de 370 registros, mientras que la ejecución obtuvo 50 registros. El Planning Time fue de 0.287 ms y el Execution Time de 78.646 ms.

Query 8: El plan realiza un Parallel Seq Scan sobre deliveries, utilizando 2 workers adicionales para procesar los registros en paralelo. Después del filtro se obtuvieron 48,453 registros, que luego fueron ordenados y agrupados mediante GroupAggregate para obtener los 7 días de la semana.

La estimación fue de 48,463 registros, mientras que la ejecución obtuvo 48,453 registros. El Planning Time fue de 0.168 ms y el Execution Time de 131.966 ms.

Query 9: El plan combina trips, routes, maintenance y vehicles mediante distintos Hash Join. Durante el procesamiento se generan 2,718,513 registros, que luego son ordenados y agrupados primero por vehículo y finalmente por tipo de vehículo, obteniendo los 4 resultados finales. El ordenamiento principal utiliza external merge y 152,816 kB de disco.

La estimación de ese procesamiento fue de 2,455,345 registros, mientras que la ejecución procesó 2,718,513 registros. El Planning Time fue de 4.435 ms y el Execution Time de 6120.501 ms.

Query 10: El plan combina trips, deliveries, drivers y routes mediante distintos Hash Join. Luego utiliza GroupAggregate y varias operaciones WindowAgg para calcular los rankings. Finalmente ordena los resultados y utiliza Limit para obtener los primeros 20 registros.

La estimación para el cálculo de los rankings fue de 133 registros, mientras que la ejecución procesó 371 registros. El Planning Time fue de 0.510 ms y el Execution Time de 162.397 ms.

3. Optimización mediante índices

Los índices permiten a PostgreSQL acceder de forma más eficiente a los datos, utilizando determinadas columnas como referencia para reducir la cantidad de registros que deben recorrerse durante una consulta. Su utilidad depende de las operaciones realizadas, como filtros, búsquedas, relaciones entre tablas o análisis temporales.

En esta etapa se implementan cinco índices, orientados específicamente a optimizar los JOIN sobre viajes, el análisis temporal de entregas, los registros de mantenimiento por vehículo, las consultas sobre conductores y las métricas asociadas a las rutas.

Los índices se crean mediante 03_optimization_indexes.sql. Posteriormente, se vuelven a medir los tiempos de ejecución para compararlos con los valores iniciales y evaluar el impacto obtenido.

Resultados

Query	Antes (ms)	Después (ms)	Diferencia (ms)	Variación
Query 1	0.160	0.093	-0.067	-41.88 %
Query 2	0.130	0.103	-0.027	-20.77 %
Query 3	25.358	23.368	-1.990	-7.85 %
Query 4	107.594	105.503	-2.091	-1.94 %
Query 5	48.132	47.163	-0.969	-2.01 %
Query 6	185.459	185.565	+0.106	+0.06 %
Query 7	78.646	68.062	-10.584	-13.46 %
Query 8	131.966	68.370	-63.596	-48.19 %
Query 9	6120.501	1309.980	-4810.521	-78.60 %
Query 10	162.397	157.531	-4.866	-3.00 %

Los resultados muestran que el impacto de los índices varía según la consulta. Las mejoras más significativas se observan en las Queries 8 y 9, cuyos tiempos de ejecución se redujeron aproximadamente un 48 % y un 79 %, respectivamente. En otras consultas la diferencia fue menor, mientras que la Query 6 presentó un tiempo prácticamente igual antes y después de la optimización.

Los tiempos obtenidos deben considerarse como una referencia del entorno utilizado, ya que pueden variar según el hardware del equipo, la carga del sistema y el estado de la memoria caché de PostgreSQL y del sistema operativo. Una consulta ejecutada nuevamente puede obtener un tiempo menor debido a que parte de los datos utilizados en la primera ejecución pueden permanecer en memoria.

Para mantener un criterio uniforme, cada medición fue realizada sobre la primera ejecución de la consulta, tanto antes como después de implementar los índices. Antes de realizar las mediciones se cerró y volvió a abrir el proyecto, evitando ejecutar previamente las consultas que serían evaluadas.

4. Posibles mejoras

A partir del análisis de los resultados se identificaron algunas oportunidades para ampliar las consultas y obtener información más útil para el negocio:

Query 1: incorporar la capacidad promedio por tipo de vehículo. El conteo permite conocer la composición de la flota, pero no refleja la capacidad operativa que aporta cada categoría.

Query 2: considerar únicamente conductores activos. La consulta busca anticipar vencimientos de licencias, por lo que incluir conductores inactivos puede generar alertas que no requieren una acción inmediata.

Query 3: enfocar el monitoreo principalmente en los viajes in_progress, separando la operación actual del histórico de viajes completados. Además, se podría incorporar un período de tiempo, por ejemplo los últimos 30 o 90 días, ya que al utilizar todo el histórico los viajes completed se acumulan y dominan el resultado.

Query 4: incorporar una medida relativa de demanda. El volumen total permite comparar ciudades, pero puede estar condicionado por diferencias en la cantidad de viajes realizados hacia cada destino.

Query 5: establecer un período de análisis común. Al utilizar todo el histórico de cada conductor, quienes llevan más tiempo operando tienen mayor posibilidad de acumular viajes, dificultando una comparación equilibrada de la carga de trabajo.

Query 6: incorporar el tiempo total de actividad de cada conductor. Las entregas por viaje permiten analizar parte de su operación, pero no consideran cuánto tiempo fue necesario para completar esos viajes y entregas.

Query 7: considerar el tipo de vehículo utilizado en cada ruta. El consumo de combustible no depende únicamente de la distancia recorrida, por lo que incorporar las características del vehículo permitiría interpretar mejor las diferencias de consumo.

Query 8: desagregar los retrasos por ciudad, ruta o franja horaria. El análisis por día de la semana permite detectar diferencias generales, pero no permite identificar dónde o en qué momento se originan los retrasos.

Query 9: incorporar la antigüedad del vehículo y el tipo de mantenimiento realizado. El costo por kilómetro permite comparar categorías, pero no permite determinar si las diferencias están relacionadas con el tipo de vehículo, su antigüedad o determinados mantenimientos.

Query 10: mejorar la forma en que se calcula el ranking final. Actualmente, el resultado se obtiene promediando las posiciones de cada conductor en puntualidad, eficiencia y productividad. Esto puede generar diferencias exageradas, ya que dos conductores con valores muy similares pueden ocupar posiciones distintas y esa diferencia termina afectando directamente el ranking final. Sería más representativo calcular el resultado utilizando los valores reales de las métricas y definir su importancia según las prioridades del negocio. Además, se podrían considerar únicamente los conductores activos para que el ranking represente al personal que forma parte de la operación actual.

Avance 3

Problema de negocio

FleetLogix cuenta con una base de datos operacional en PostgreSQL que permite almacenar y consultar la información generada por sus operaciones. Sin embargo, esta estructura está orientada principalmente a la gestión transaccional y no fue diseñada para concentrar información histórica preparada específicamente para el análisis.

En este tercer avance se busca separar el entorno operacional del analítico mediante la construcción de un Data Warehouse en Snowflake. A su vez, se desarrolla un pipeline ETL que permite extraer los datos desde PostgreSQL, transformarlos según las necesidades del modelo dimensional y cargarlos en Snowflake de forma periódica.

De esta manera, los datos generados durante la operación pueden incorporarse progresivamente al entorno analítico sin depender de cargas manuales.

Recursos proporcionados

- A3-04_dimensional_model.sql: script base utilizado para implementar el modelo dimensional y configurar la estructura del Data Warehouse en Snowflake.

- `A3-05_etl_pipeline_estudiantes.py`: script Python utilizado para desarrollar el pipeline encargado de extraer, transformar y cargar diariamente los datos desde PostgreSQL hacia Snowflake.
- `pasos_para_usar_Snowflake.pdf`: guía utilizada como referencia para la configuración inicial del entorno de trabajo en Snowflake.

Como complemento al pipeline diario, se desarrolló ***initial_load.py***, utilizado para realizar la carga inicial de los datos históricos existentes en PostgreSQL y dejar poblado el Data Warehouse antes de comenzar con las cargas incrementales.

Desarrollo

El tercer avance parte de la base operacional construida durante los avances anteriores y agrega una nueva capa destinada específicamente al análisis. Para ello, primero se implementó en Snowflake un modelo dimensional con esquema estrella, separando las métricas de las entregas de las distintas dimensiones utilizadas para analizarlas.

Una vez creada esta estructura, se realizó una carga inicial de los datos históricos disponibles en PostgreSQL. Esta etapa permitió comenzar a trabajar con un Data Warehouse poblado y comprobar el funcionamiento de las relaciones entre las dimensiones y la tabla de hechos.

Finalmente, se completó el pipeline ETL en Python para automatizar las cargas posteriores. El proceso consulta las entregas correspondientes al día anterior, aplica las transformaciones y controles de calidad necesarios y carga los registros resultantes en Snowflake. También incorpora el tratamiento de cambios históricos y el cálculo anticipado de totales destinados a facilitar análisis posteriores.

El trabajo se organizó en tres etapas principales:

- Implementación del Data Warehouse: creación del modelo estrella y configuración del entorno analítico en Snowflake.
- Carga inicial: transferencia y transformación de los datos históricos desde PostgreSQL mediante `initial_load.py`.
- Pipeline ETL diario: automatización de la extracción, transformación y carga incremental mediante `A3-05_etl_pipeline_estudiantes.py`.

1. Implementación del Data Warehouse

Para separar el procesamiento operacional del análisis de la información, se creó un Data Warehouse en Snowflake utilizando como origen los datos almacenados en PostgreSQL. La estructura fue implementada mediante **`A3-04_dimensional_model.sql`**, tomando como base las entidades y relaciones desarrolladas durante el primer avance.

En Snowflake se creó la base de datos **`FLEETLOGIX_DW`** y dentro de ella el esquema **`ANALYTICS`**, destinado a concentrar las tablas y objetos utilizados para el análisis. También se configuró el warehouse **`FLEETLOGIX_WH`**, encargado de proporcionar los recursos de cómputo necesarios para ejecutar las consultas y procesos de carga.

A diferencia del modelo relacional utilizado en PostgreSQL, la nueva estructura se organizó mediante un **esquema estrella**, tomando las entregas completadas como centro del análisis. Esto permite concentrar las principales métricas de cada entrega y relacionarlas con diferentes dimensiones del negocio.

1.1 Modelo dimensional

El modelo tiene como tabla central ***fact_deliveries***, donde cada registro representa una entrega completada. En ella se almacenan las claves que permiten relacionar cada entrega con sus dimensiones y métricas como distancia recorrida, combustible consumido, tiempo de entrega, retraso, eficiencia de combustible, costo e ingreso estimado.

La tabla de hechos se relaciona con seis dimensiones:

- `dim_date`: incorpora la información del calendario necesaria para realizar análisis temporales.

- dim_time: permite analizar las entregas según la hora del día.
- dim_vehicle: conserva las características de los vehículos utilizados en las operaciones.
- dim_driver: almacena la información de los conductores y permite conservar cambios históricos relevantes.
- dim_route: contiene las características de las rutas utilizadas en los viajes.
- dim_customer: permite identificar y analizar la actividad asociada a los clientes.

Esta estructura permite consultar las métricas de las entregas desde distintas perspectivas sin realizar el análisis directamente sobre las tablas operacionales de PostgreSQL.

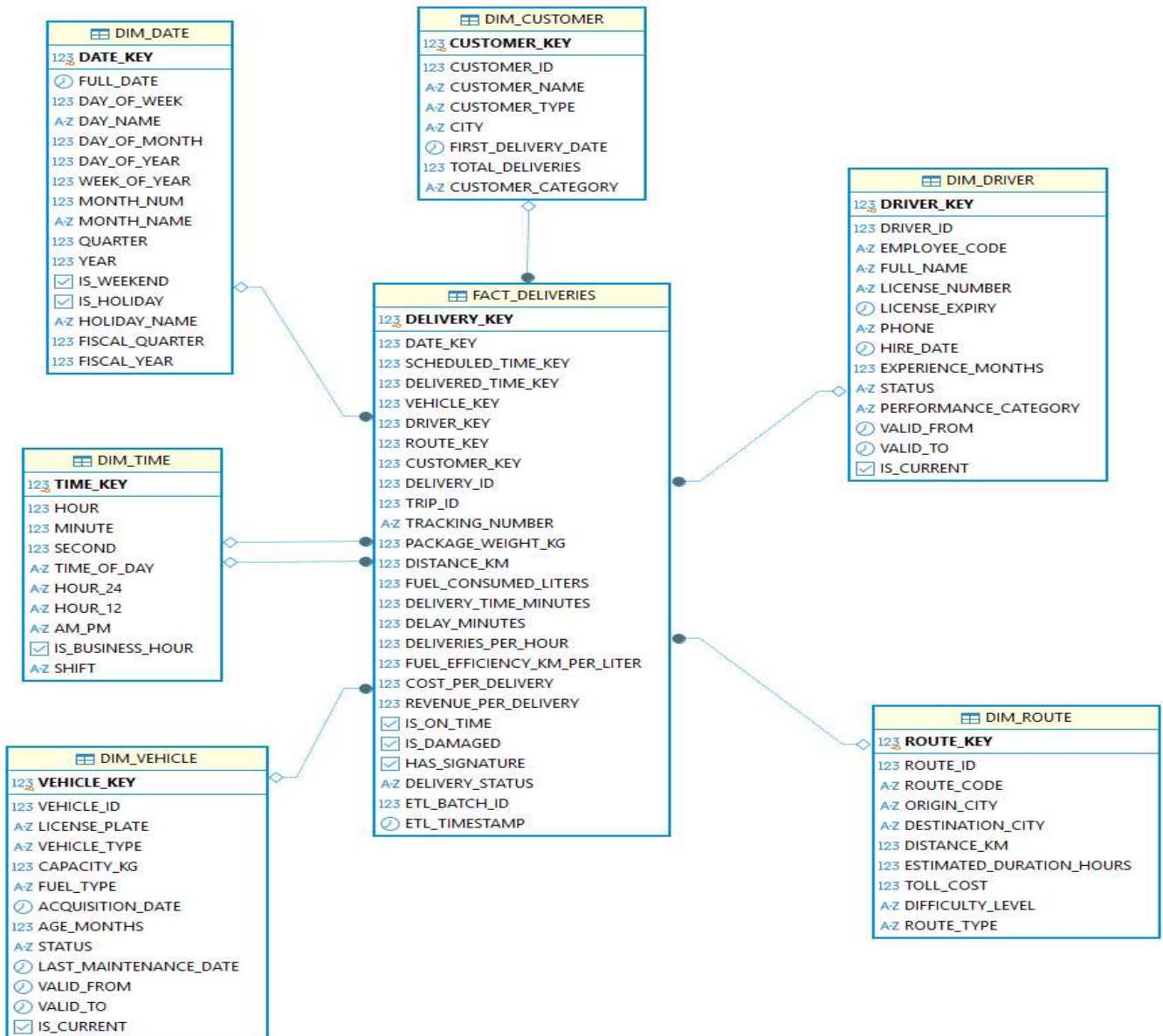


Figura 2. Modelo dimensional implementado en Snowflake para FleetLogix.

2. Carga inicial de datos históricos

Una vez creado el modelo dimensional en Snowflake, fue necesario poblarlo con la información histórica almacenada en PostgreSQL. El pipeline ETL proporcionado está diseñado para procesar las entregas del día anterior, por lo que no permite realizar por sí solo esta primera carga completa.

Para resolverlo se desarrolló `initial_load.py`, un script complementario que extrae los datos históricos de PostgreSQL y los adapta a la estructura del Data Warehouse. Primero se cargan las dimensiones de fecha, hora, vehículos, conductores, rutas y clientes. Luego se cargan las entregas completadas en `fact_deliveries`, la tabla central del modelo dimensional donde se concentran las métricas utilizadas para el análisis.

Durante esta transformación se calculan valores como el tiempo de entrega, el retraso respecto del horario programado, las entregas por hora, la eficiencia de combustible y los valores estimados de costo e ingreso. Esta carga se realiza una única vez para establecer la base histórica del Data Warehouse. A partir de allí, las nuevas entregas se incorporan mediante el pipeline ETL diario.

2.1 Resultado de la carga inicial

Una vez finalizada la ejecución, se verificó en Snowflake la cantidad de registros cargados en cada una de las dimensiones y en la tabla de hechos.

Tabla	Registros cargados
DIM_DATE	731
DIM_TIME	24
DIM_VEHICLE	200
DIM_DRIVER	400
DIM_ROUTE	50
DIM_CUSTOMER	341.378
FACT_DELIVERIES	397.882

3. Pipeline ETL diario

Una vez realizada la carga histórica, se completó A3-05_etl_pipeline_estudiantes.py para automatizar la incorporación de nuevas entregas al Data Warehouse. El pipeline conecta la base operacional PostgreSQL (fleetlogix) con el entorno analítico Snowflake (FLEETLOGIX_DW.ANALYTICS), procesando únicamente las entregas correspondientes al día anterior.

El proceso se organiza en las tres etapas propias de un flujo ETL: extracción, transformación y carga.

3.1 Extracción de datos

La función `extract_daily_data()` consulta PostgreSQL y obtiene las entregas completadas durante el período diario. Para construir el conjunto de datos combina información de las tablas operacionales `deliveries`, `trips` y `routes`.

De esta extracción se obtienen datos como `delivery_id`, `trip_id`, `customer_name`, `package_weight_kg`, `scheduled_datetime` y `delivered_datetime`, junto con información del viaje como `vehicle_id`, `driver_id`, `route_id`, `departure_datetime`, `arrival_datetime` y `fuel_consumed_liters`.

Al limitar la consulta al día anterior, el pipeline funciona como una carga incremental, evitando volver a procesar todo el historial en cada ejecución.

3.2 Transformación y controles de calidad

La función `transform_data()` prepara los registros extraídos antes de enviarlos a Snowflake. A partir de las fechas del viaje y de la entrega se calculan dos métricas diferentes: `delivery_time_minutes`, que representa el tiempo transcurrido desde `departure_datetime` hasta `delivered_datetime`, y `delay_minutes`, que mide el retraso respecto de `scheduled_datetime`.

También se generan métricas analíticas como `deliveries_per_hour`, `fuel_efficiency_km_per_liter`, `cost_per_delivery` y `revenue_per_delivery`, además de `is_on_time` para identificar las entregas realizadas dentro del margen establecido.

Antes de realizar la carga se aplican controles de calidad sobre los registros. Se descartan tiempos de entrega negativos y pesos fuera de los rangos definidos, mientras que una entrega realizada antes de `scheduled_datetime` se conserva como válida y recibe un `delay_minutes` igual a cero.

3.3 Actualización de dimensiones

Antes de insertar las nuevas entregas, `load_dimensions()` actualiza las dimensiones necesarias en Snowflake. Los clientes se incorporan en `DIM_CUSTOMER`, mientras que `DIM_DRIVER` contempla cambios históricos mediante una estrategia SCD Tipo 2.

Esto permite que un cambio de estado de un conductor no sobrescriba directamente su información anterior. Los campos `valid_from`, `valid_to` e `is_current` permiten conservar las distintas versiones y determinar cuál se encuentra vigente.

3.4 Carga de entregas y métricas

La función `load_facts()` incorpora los registros transformados en `FACT_DELIVERIES`, tabla central del modelo dimensional. Cada entrega queda relacionada mediante sus claves con `DIM_DATE`, `DIM_TIME`, `DIM_VEHICLE`, `DIM_DRIVER`, `DIM_ROUTE` y `DIM_CUSTOMER`.

Junto con los datos identificativos de la entrega se almacenan las métricas calculadas durante la transformación. Cada ejecución también genera un `etl_batch_id`, utilizado para identificar los registros pertenecientes a un mismo proceso de carga.

Finalmente, `_calculate_daily_totals()` genera información resumida en `DAILY_TOTALS`, dejando precalculados los totales correspondientes al lote procesado.

3.5 Ejecución automática

El proceso completo es coordinado por `run_etl()`, que administra la conexión con ambas bases, la extracción, transformación y carga de los datos. La ejecución queda programada mediante `schedule` para realizarse diariamente a las 02:00.

De esta forma, la funcionalidad final del pipeline puede resumirse como:

PostgreSQL (fleetlogix) → extracción de entregas diarias → transformación y controles de calidad → actualización de dimensiones → `FACT_DELIVERIES` → `DAILY_TOTALS` en Snowflake.

3.6 Modificaciones realizadas al pipeline proporcionado

El archivo `A3-05_etl_pipeline_estudiantes.py` fue utilizado como base para el desarrollo del pipeline diario. Sobre esta estructura se completaron y ajustaron las funciones necesarias para adaptarlo al modelo dimensional implementado en Snowflake.

Uno de los principales ajustes se realizó sobre las métricas de tiempo. Se diferenció `delivery_time_minutes`, calculado desde `departure_datetime` hasta `delivered_datetime`, de `delay_minutes`, que compara `delivered_datetime` con `scheduled_datetime`. Esto permite distinguir la duración real de una entrega de su retraso respecto del horario programado y conservar correctamente las entregas anticipadas.

También se completó la lógica necesaria para relacionar los registros con las dimensiones del Data Warehouse, incluyendo la obtención de las claves de `DIM_CUSTOMER` y el tratamiento de cambios históricos en `DIM_DRIVER` mediante `valid_from`, `valid_to` e `is_current`.

Finalmente, se completó la carga de las entregas y métricas en `FACT_DELIVERIES`, la identificación de cada ejecución mediante `etl_batch_id` y la generación de los totales diarios almacenados en `DAILY_TOTALS`.

4. Pruebas y validación

Una vez completado el pipeline, se realizaron ejecuciones de prueba para verificar la conexión entre PostgreSQL y Snowflake y comprobar el funcionamiento completo del proceso ETL.

Debido a que la base operacional contiene datos históricos y no se generan nuevas entregas diariamente, para la prueba se utilizó temporalmente una fecha con entregas existentes. Esto permitió comprobar que `extract_daily_data()` recuperara los registros correspondientes, que `transform_data()` aplicara las métricas y controles definidos y que posteriormente `load_dimensions()` y `load_facts()` incorporaran la información en Snowflake.

Durante las pruebas también se revisaron los cálculos de `delivery_time_minutes` y `delay_minutes`. Esta validación permitió ajustar el cálculo del tiempo de entrega y el control de valores negativos, aplicando este último antes del redondeo de los datos.

La validación final se realizó consultando las tablas del esquema `FLEETLOGIX_DW.ANALYTICS`, comprobando la actualización de las dimensiones, la incorporación de las entregas en `FACT_DELIVERIES` y la generación de los totales en `DAILY_TOTALS`.

5. Posibles mejoras

Si bien el pipeline permite automatizar la carga diaria de información desde PostgreSQL hacia Snowflake, existen algunos aspectos que podrían mejorarse en futuras versiones.

Una posible mejora sería optimizar la carga de grandes volúmenes de datos. Durante la carga histórica se observó que la inserción de cientos de miles de registros puede requerir un tiempo considerable, por lo que podrían evaluarse métodos de carga más eficientes.

También sería conveniente incorporar un mayor control para evitar cargas duplicadas si un mismo período se procesa más de una vez, utilizando la fecha procesada o el `etl_batch_id` para verificar previamente los registros existentes.

Por último, podrían ampliarse los controles y registros de ejecución, facilitando la identificación de errores y el seguimiento de cada carga realizada.

Avance 4

Introducción

Este avance se centra en la comprensión y documentación de una arquitectura cloud, sin requerir su implementación. Debido a que los servicios de AWS pueden generar costos, el trabajo se realiza a partir de la arquitectura y los recursos proporcionados, analizando su funcionamiento y las decisiones de diseño necesarias.

Problema de negocio

FleetLogix necesita modernizar su infraestructura tecnológica para poder procesar información de la flota en tiempo real. El sistema debe recibir los eventos generados durante las entregas, consultar su estado, estimar tiempos de llegada y detectar posibles desvíos de las rutas planificadas.

Para responder a estas necesidades se plantea el diseño de una arquitectura cloud que permita integrar el procesamiento de estos eventos con el almacenamiento de información operativa e histórica. La solución también debe contemplar la disponibilidad de la base de datos PostgreSQL en la nube, mecanismos de respaldo, monitoreo y medidas básicas de seguridad.

Recursos proporcionados

Para el desarrollo del cuarto avance se proporcionan los siguientes recursos:

- **aws_architecture_diagram.png**: diagrama que muestra la arquitectura AWS propuesta para FleetLogix y cómo se conectan sus distintos componentes.
- **lambda_functions.py**: script Python que contiene tres funciones Lambda para verificar entregas, calcular el ETA de los vehículos y detectar desvíos de las rutas.
- **aws_setup.py**: script Python que configura los recursos principales de AWS, incluyendo RDS PostgreSQL, S3, DynamoDB, backups y permisos IAM. También incluye el proceso para migrar la base de datos PostgreSQL local hacia RDS.
- **Entregables y paso a paso AWS – Avance 4.pdf**: guía con los entregables del avance y los pasos necesarios para implementar la arquitectura en AWS en caso de querer realizar la parte práctica opcional.

Desarrollo

La arquitectura propuesta permite analizar cómo los distintos servicios de AWS se conectan para recibir, procesar y almacenar la información generada durante la operación de FleetLogix.

El desarrollo comprende el análisis de la arquitectura y del flujo de información, el funcionamiento de los recursos definidos en los scripts proporcionados y las decisiones relacionadas con almacenamiento, procesamiento, monitoreo, seguridad y costos.

1. Arquitectura AWS propuesta

La arquitectura proporcionada plantea que la aplicación móvil de FleetLogix se comuniquen con API Gateway, que funciona como punto de entrada para las solicitudes. Según el tipo de operación, la solicitud se dirige hacia una de las funciones Lambda encargadas de procesarla.

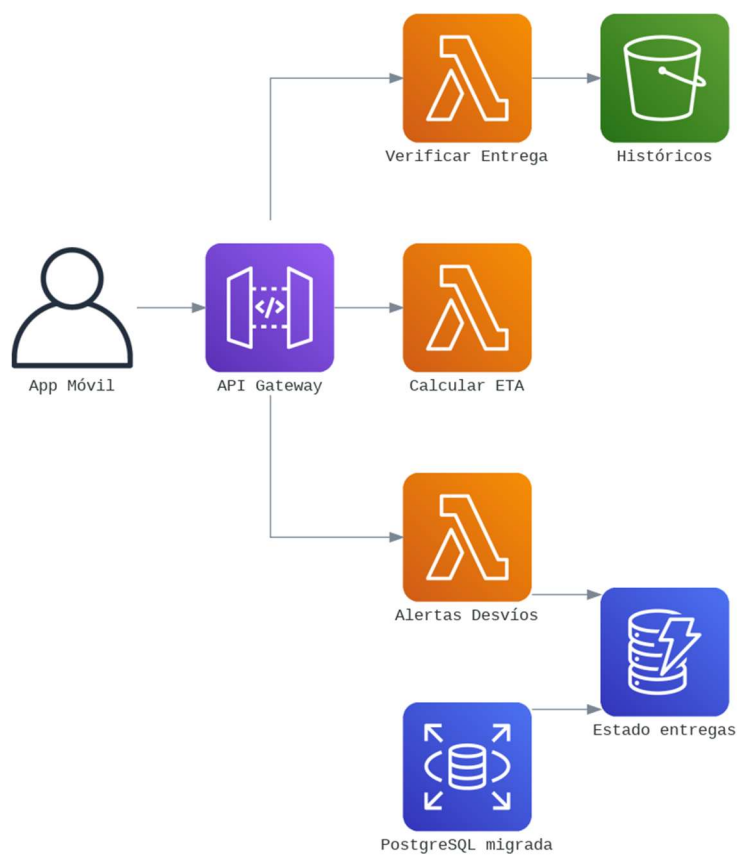
Se definen tres funciones Lambda con tareas diferentes: verificar una entrega, calcular el ETA de un vehículo y detectar desvíos de la ruta esperada. De esta manera, cada función se ocupa de una tarea específica y puede ejecutarse cuando se produce el evento correspondiente.

Para almacenar la información se utilizan distintos servicios según la necesidad. RDS PostgreSQL mantiene la base de datos relacional, DynamoDB almacena información operativa que requiere consultas y actualizaciones rápidas, y S3 se utiliza para conservar archivos e información histórica.

La combinación de estos componentes permite separar la recepción de las solicitudes, su procesamiento y el almacenamiento de los datos, manteniendo cada tarea asociada al servicio que corresponde.

1.1 Diagrama de arquitectura

El siguiente diagrama representa la arquitectura AWS propuesta para FleetLogix y muestra cómo se relacionan los servicios utilizados para recibir las solicitudes, procesar los eventos y almacenar la información.



1.2 Flujo de información

La arquitectura organiza el flujo de información en distintas etapas. Los datos ingresan al sistema desde la aplicación móvil y son recibidos por API Gateway, que funciona como punto de entrada y dirige cada solicitud hacia la función Lambda correspondiente.

Una vez recibida la solicitud, la función Lambda procesa la información y se comunica con los servicios de almacenamiento necesarios. Según el tipo de dato, la información puede mantenerse como estado actual en DynamoDB, almacenarse en la base relacional PostgreSQL de RDS o conservarse como información histórica en S3.

Además de las solicitudes iniciadas desde la aplicación, algunas funciones pueden ejecutarse automáticamente mediante eventos o intervalos programados, permitiendo procesar nueva información a medida que se genera.

2. Recursos de AWS

El script `aws_setup.py` reúne la configuración necesaria para crear los principales recursos que utiliza la arquitectura. Aunque estos recursos no se implementan durante el avance, el código permite analizar cómo se configuraría cada servicio y qué función cumpliría dentro de FleetLogix.

2.1 Amazon RDS PostgreSQL

La función `crear_rds_postgresql()` define una instancia de Amazon RDS con PostgreSQL, destinada a alojar en AWS la base de datos relacional que actualmente funciona de manera local.

La instancia se configura con PostgreSQL 15.4, 20 GB de almacenamiento y una política de backups automáticos con una retención de 7 días. De esta manera, FleetLogix puede mantener su modelo relacional en PostgreSQL, pero utilizando un servicio administrado por AWS para alojar la base de datos y gestionar parte de su infraestructura.

2.2 Amazon S3

La función `crear_s3_bucket()` define un bucket de Amazon S3 llamado `fleetlogix-data`, destinado al almacenamiento de archivos y datos que no necesitan mantenerse dentro de una base de datos relacional.

El almacenamiento se organiza en cuatro carpetas: `raw-data` para los datos originales, `processed-data` para los datos procesados, `backups` para las copias de respaldo y `logs` para los registros generados por los procesos.

Además, el script establece una regla para que los archivos almacenados en `raw-data` pasen a S3 Glacier después de 90 días. Esto permite conservar información histórica que ya no necesita un acceso frecuente utilizando una alternativa de almacenamiento pensada para archivos de largo plazo.

2.3 Amazon DynamoDB

La función `crear_tablas_dynamodb()` define cuatro tablas en Amazon DynamoDB, utilizadas para almacenar información que necesita consultarse o actualizarse rápidamente durante la operación.

Las tablas creadas son:

- `deliveries_status`: almacena el estado actual de las entregas.
- `vehicle_tracking`: registra la ubicación y el seguimiento de los vehículos a lo largo del tiempo.
- `routes_waypoints`: almacena los puntos que forman las rutas esperadas y que luego se utilizan para detectar posibles desvíos.
- `alerts_history`: conserva las alertas generadas cuando se detecta un desvío.

Las cuatro tablas utilizan el modo PAY_PER_REQUEST, por lo que no se define una capacidad fija de lectura y escritura. En este caso, DynamoDB complementa a PostgreSQL almacenando información operativa que debe estar disponible rápidamente para las funciones Lambda.

2.4 Backups automáticos

La función `configurar_backups_automaticos()` complementa la configuración de respaldos de la base PostgreSQL alojada en RDS. La instancia se configura para mantener backups automáticos durante 7 días, permitiendo recuperar información ante una pérdida o un problema con la base de datos.

Además, la función crea un snapshot inicial de RDS, que funciona como una copia puntual del estado de la base. Este snapshot es independiente de los backups automáticos y queda almacenado hasta que se decida eliminarlo.

La retención de 7 días se aplica a los backups automáticos y no a los datos almacenados en PostgreSQL. Por lo tanto, los registros de la base continúan existiendo normalmente aunque tengan una antigüedad mayor a ese período.

2.5 Rol IAM para las funciones Lambda

La función `crear_rol_iam_lambda()` define un rol de IAM llamado `FleetLogixLambdaRole`, que establece los permisos que tendrán las funciones Lambda para trabajar con otros servicios de AWS.

El script permite que las Lambda generen sus registros de ejecución y accedan a DynamoDB, S3 y SNS. Estos permisos son necesarios porque las funciones deben consultar y guardar información durante su ejecución y, en el caso de los desvíos de ruta, enviar una alerta mediante SNS.

La configuración proporcionada utiliza permisos amplios (`FullAccess`) para estos servicios. En una implementación real sería conveniente limitar los permisos a las acciones y recursos que cada función necesita, siguiendo el principio de mínimo privilegio.

2.6 Migración de PostgreSQL hacia RDS

La función `migrar_datos_postgresql()` prepara el proceso necesario para trasladar la base de datos PostgreSQL utilizada localmente hacia la instancia de RDS. La migración no se ejecuta directamente desde Python, sino que la función genera un archivo llamado `migrate_to_rds.sh` con los comandos necesarios para realizarla.

El proceso comienza utilizando `pg_dump` para generar una copia de la base de datos local de FleetLogix en el archivo `fleetlogix_dump.sql`. Luego se utiliza `psql` para conectarse a la instancia PostgreSQL alojada en RDS, crear allí la base de datos `fleetlogix` y restaurar el contenido del archivo generado previamente.

De esta manera, el procedimiento permite conservar la estructura y los datos existentes en PostgreSQL y trasladarlos al entorno de AWS, sin tener que reconstruir la base desde cero. Una vez completada la migración, RDS pasa a alojar la base relacional utilizada por FleetLogix.

3. Funciones Lambda

El archivo `lambda_functions.py` contiene tres funciones Lambda que se encargan de procesar distintas situaciones de la operación de FleetLogix. Cada función recibe la información necesaria para realizar una tarea específica y puede consultar o actualizar los datos almacenados en DynamoDB.

3.1 Verificación de entregas

La función `lambda_verificar_entrega()` permite consultar si una entrega fue completada. Para realizar la búsqueda recibe el `delivery_id` y, si está disponible, el número de seguimiento (`tracking_number`).

La función consulta la tabla `deliveries_status` de DynamoDB utilizando el identificador de la entrega. Si encuentra el registro, comprueba su campo `status`: cuando el valor es `delivered`, considera que la entrega fue completada. Como respuesta devuelve el identificador, número de seguimiento, estado de la entrega, fecha de entrega y un indicador que informa si se encuentra completada.

El código también contempla situaciones en las que la consulta no puede realizarse correctamente. Si no se recibe un `delivery_id`, devuelve un error 400; si la entrega no existe en DynamoDB, devuelve 404; y si ocurre un error durante la ejecución, devuelve 500. Cuando la consulta se procesa correctamente, la respuesta utiliza el código 200.

Esta función permite que el estado actual de una entrega pueda consultarse sin tener que recorrer la base relacional completa, utilizando la información operativa almacenada en DynamoDB.

3.2 Cálculo de ETA

La función `lambda_calcular_eta()` permite estimar cuánto tiempo falta para que un vehículo llegue a su destino. Para realizar el cálculo recibe el identificador del vehículo, su ubicación actual, el destino y la velocidad a la que se desplaza.

A partir de las coordenadas de la ubicación actual y del destino, la función calcula una distancia aproximada en línea recta. Luego utiliza esa distancia junto con la velocidad del vehículo para estimar el tiempo restante y obtener una fecha y hora aproximada de llegada.

Una vez realizado el cálculo, la información se guarda en la tabla `vehicle_tracking` de DynamoDB. El registro incluye el vehículo, la ubicación actual, el destino, la distancia restante, la velocidad, el ETA calculado y el momento en que se realizó la actualización.

El cálculo utilizado es una aproximación simple y no considera factores como el tránsito, el estado de las calles o el recorrido real de la ruta. Aun así, permite representar cómo podría generarse y actualizarse automáticamente una estimación de llegada a partir de la información enviada por los vehículos.

3.3 Detección de desvíos de ruta

La función `lambda_alerta_desvio()` permite detectar si un vehículo se encuentra demasiado alejado de la ruta que debería estar siguiendo. Para realizar esta verificación recibe la ubicación actual del vehículo, su identificador, la ruta asignada y el conductor.

La función consulta en la tabla `routes_waypoints` de DynamoDB los puntos que representan la ruta esperada y calcula la distancia aproximada entre la ubicación actual del vehículo y esos puntos. Luego toma como referencia el punto más cercano para determinar qué tan alejado se encuentra el vehículo de su recorrido.

El script establece un límite de 5 kilómetros. Si la distancia al punto más cercano de la ruta supera ese valor, el vehículo se considera desviado. En ese caso se genera una alerta mediante Amazon SNS con información del vehículo, conductor, ruta, ubicación y momento en que se detectó el desvío. La alerta también se registra en la tabla `alerts_history` de DynamoDB para conservar un historial de estos eventos.

La función permite detectar automáticamente que un vehículo dejó de seguir la ruta esperada, aunque no determina la causa del desvío. Su objetivo es identificar la situación y generar la alerta para que pueda ser revisada.

4. Procesamiento automático de eventos

Las funciones Lambda no necesitan ejecutarse manualmente. AWS permite asociarlas a distintos eventos para que se activen cuando ocurre una determinada situación dentro del sistema.

La función de verificación de entregas se plantea conectada a API Gateway mediante una solicitud POST `/deliveries/verify`. De esta forma, puede ejecutarse cuando desde la aplicación se solicita verificar el estado de una entrega.

Para el cálculo de ETA, el script propone utilizar EventBridge para ejecutar el proceso automáticamente cada 5 minutos. Esto permite actualizar periódicamente la estimación de llegada de los vehículos que se encuentran en ruta.

En el caso de la detección de desvíos, se propone utilizar Kinesis para recibir las actualizaciones de ubicación GPS. Cada nueva posición puede activar la función Lambda, que compara la ubicación del vehículo con su ruta y genera una alerta si detecta un desvío superior al límite establecido.

De esta manera, la arquitectura combina solicitudes realizadas desde la aplicación con procesos automáticos que se ejecutan a partir de horarios o nuevos datos generados durante la operación.

5. Monitoreo y observabilidad

Para supervisar el funcionamiento de la arquitectura se propone utilizar Amazon CloudWatch, que permite recopilar métricas y registros de los distintos servicios de AWS. Esto permite detectar errores y conocer el comportamiento del sistema sin tener que revisar cada componente de forma individual.

En las funciones Lambda se pueden controlar métricas como la cantidad de ejecuciones, los errores y el tiempo que tarda cada función en responder. En API Gateway resulta útil observar las solicitudes recibidas, los errores y los tiempos de respuesta. Para RDS se pueden monitorear aspectos como el uso de CPU, las conexiones a PostgreSQL y el espacio de almacenamiento disponible.

También se pueden configurar alarmas automáticas cuando una métrica supera determinados límites. Por ejemplo, una cantidad elevada de errores en las funciones Lambda podría generar una alerta para informar al equipo y permitir que el problema sea revisado.

De esta forma, CloudWatch permite conocer el estado general de la arquitectura y detectar problemas que podrían afectar el procesamiento de la información o la disponibilidad de los servicios.

6. Seguridad

La arquitectura debe contemplar medidas básicas de seguridad para proteger tanto los datos como el acceso a los distintos servicios de AWS.

El acceso a los recursos se controla mediante IAM, utilizando usuarios y roles con permisos definidos y evitando el uso de la cuenta root para las tareas habituales. En el caso de las funciones Lambda, el script crea un rol específico para permitir su comunicación con servicios como DynamoDB, S3 y SNS.

También se debe mantener la encriptación de los datos almacenados, especialmente en servicios como RDS, DynamoDB y S3. De esta forma, la información permanece protegida aunque se encuentre almacenada dentro de la infraestructura de AWS.

El script proporcionado utiliza permisos FullAccess para algunos servicios y configura la instancia RDS como públicamente accesible. Estas configuraciones simplifican el ejemplo, pero en una implementación real sería conveniente limitar los permisos a las acciones necesarias y restringir el acceso a la base de datos.

7. Costos de la arquitectura

La arquitectura debe mantenerse dentro de un costo teórico inferior a USD 50 mensuales. Para realizar esta estimación se deben considerar principalmente los servicios utilizados, el volumen de información almacenada y la cantidad de solicitudes o ejecuciones que recibe cada componente.

Servicios como Lambda y DynamoDB permiten que el costo dependa en gran medida del uso realizado, mientras que S3 depende principalmente del volumen almacenado y de las operaciones sobre los archivos. En el caso de RDS, el costo está asociado a mantener disponible la instancia PostgreSQL definida para la base de datos.

Como la arquitectura no se implementa realmente, el costo se plantea a partir de un escenario teórico de uso para FleetLogix. La estimación permite comprobar si los recursos seleccionados pueden mantenerse dentro del presupuesto establecido sin necesidad de desplegarlos en AWS.

Para realizar la estimación se utilizó [AWS Pricing Calculator](#), configurando los servicios en la región US East (N. Virginia), la misma definida en los scripts proporcionados. Para los valores de uso que no se encuentran definidos en el código se utilizaron supuestos teóricos de funcionamiento.

Servicio	Costo mensual
Amazon RDS for PostgreSQL	USD 15,44
AWS Lambda	USD 0,00
Amazon API Gateway	USD 0,50
Amazon DynamoDB	USD 0,59
Amazon S3	USD 0,32
Amazon CloudWatch	USD 3,80
Amazon SNS	USD 0,00
Amazon EventBridge	USD 0,00
Amazon Kinesis Data Streams	USD 29,50
Total mensual	USD 50,15

El costo mensual estimado de la arquitectura es de USD 50,15. El servicio con mayor impacto en el total es Amazon Kinesis Data Streams, con un costo de USD 29,50 mensuales. Sin considerar este servicio, el resto de los componentes representa un costo aproximado de USD 20,65 mensuales.

El resultado supera por USD 0,15 el objetivo teórico de USD 50 mensuales. Debido a que Kinesis se utiliza para procesar de forma continua las actualizaciones de ubicación GPS, en una implementación real se podría revisar la frecuencia con la que se envían estos datos o evaluar una alternativa de procesamiento que permita reducir su costo.

La estimación anual obtenida mediante AWS Pricing Calculator es de USD 601,82, incluyendo un costo inicial de USD 0,02 asociado a S3. Los valores obtenidos corresponden al escenario teórico utilizado para este análisis y pueden variar según el uso real de los servicios.

8. Conclusión

El análisis realizado permite plantear una arquitectura cloud para FleetLogix capaz de recibir y procesar información generada durante la operación de la flota. La solución combina distintos servicios de AWS para separar la recepción de solicitudes, el procesamiento de eventos, el almacenamiento de datos y el monitoreo del sistema.

Las funciones Lambda permiten resolver tareas específicas como la verificación de entregas, el cálculo de ETA y la detección de desvíos, mientras que RDS PostgreSQL, DynamoDB y S3 cubren distintas necesidades de almacenamiento. A su vez, servicios como CloudWatch, SNS, EventBridge y Kinesis permiten incorporar monitoreo, alertas y procesamiento automático de eventos.

La estimación realizada con AWS Pricing Calculator permitió analizar también el impacto económico de la solución. El escenario planteado alcanza un costo aproximado de USD 50,15 mensuales, siendo Kinesis Data Streams el componente con mayor impacto en el total y un posible punto de optimización en una implementación real.

De esta manera, el avance permite comprender cómo podría trasladarse la infraestructura de FleetLogix hacia un entorno cloud y qué decisiones deberían considerarse antes de realizar una implementación real en AWS.